

SOFTWARE REQUIREMENTS SPECIFICATION

Platform PasarKita

Marketplace Digital UMKM

Atribut	Nilai
Nama Dokumen	Software Requirements Specification (SRS) Platform PasarKita
Versi	1.0 – Initial Requirements Baseline
Tanggal	24 Juni 2026
Sistem	Marketplace PasarKita
Pemilik Produk	Kelompok 2 RPL 2 / Universitas Logistik dan Bisnis Internasional
Target Pembaca	Product owner, developer, QA engineer, maintainer, dan dosen pengampu
Status	Dokumen kerja untuk baseline requirement dan validasi implementasi
Basis Penyusunan SRS ini disusun dari kode dan konfigurasi aplikasi PasarKita yang aktif: frontend Next.js App Router (frontend/), backend REST API Express (backend/src/), skema database Supabase PostgreSQL (backend/database/schema/000_full_schema.sql), serta integrasi dengan SmartBank dan LogistiKita. Struktur dokumen mengikuti pola SRS yang lazim digunakan tim engineering: tujuan, scope, konteks sistem, aktor, constraint, kebutuhan spesifik, interface, data, business rules, risiko, acceptance criteria, dan traceability.	

Daftar Isi

Bagian	Isi
1	Pendahuluan dan Konteks Dokumen
2	Gambaran Produk dan Batas Sistem
3	Konteks Operasional dan Arsitektur
4	Domain Data dan Aturan Bisnis
5	Kebutuhan Fungsional
6	Kebutuhan Non-Fungsional
7	Antarmuka Eksternal
8	Workflow Operasional
9	Risiko, Kontrol, dan Acceptance Criteria
10	Matriks Ketertelusuran
Lampiran	Glosarium, Referensi Internal, dan Catatan Perubahan Versi

1. Pendahuluan dan Konteks Dokumen

1.1 Tujuan Dokumen

Dokumen ini mendefinisikan kebutuhan perangkat lunak untuk PasarKita sebagai komponen *Demand Generator* B2C dalam ekosistem *microservices* UMKM RPL 2.

SRS ini berfungsi sebagai *baseline* bersama antara pemilik produk, pengembang, penguji, dan pengelola sistem. Dokumen tidak dimaksudkan sebagai uraian konseptual, melainkan sebagai spesifikasi yang menerjemahkan perilaku aplikasi ke dalam kebutuhan yang dapat dibangun, diuji, dan dipelihara. Setiap kebutuhan dirumuskan agar memiliki ruang lingkup yang jelas, dasar implementasi yang dapat ditelusuri, dan kriteria penerimaan yang dapat diverifikasi.

Rujukan format yang digunakan adalah praktik SRS yang umum dipakai di lingkungan pengembangan perangkat lunak: dokumen dimulai dari tujuan dan *scope*, dilanjutkan dengan *product perspective*, *user classes*, *constraints*, asumsi, kebutuhan fungsional, kebutuhan non-fungsional, *external interfaces*, *data requirements*, *business rules*, *acceptance criteria*, dan *traceability matrix*. Dengan struktur tersebut, dokumen dapat dipakai sebagai kontrak kerja teknis tanpa bergantung pada penjelasan lisan.

1.2 Ruang Lingkup Produk

PasarKita adalah platform *marketplace* digital B2C yang memfasilitasi transaksi produk UMKM antara *seller* dan *buyer*. Sistem berjalan sebagai aplikasi web *fullstack* dengan *frontend* Next.js 16.2 (di-deploy ke `pasarkita.vercel.app`) dan *backend* Express.js berbasis *serverless* (di-deploy ke `pasarkita-api.vercel.app`), menggunakan Supabase PostgreSQL sebagai *database*.

Dalam arsitektur ekosistem, PasarKita berperan sebagai **Demand Generator**—inisiator transaksi di sisi permintaan. Semua transaksi keuangan diproses oleh SmartBank (Kelompok 1), semua komunikasi antar-service melewati API Gateway (Kelompok 7), dan pengiriman dikoordinasikan oleh LogistiKita (Kelompok 5). PasarKita tidak mengambil alih fungsi pembayaran, *routing*, atau logistik dari *service partner* tersebut.

Sistem PasarKita bertanggung jawab pada siklus transaksi *marketplace*: registrasi/autentikasi *user*, pengelolaan katalog produk UMKM, keranjang belanja, proses *checkout* dengan kalkulasi *fee*, koordinasi pembayaran ke SmartBank, *trigger* pengiriman ke LogistiKita, manajemen status *order*, dan *dashboard analytics* untuk *superadmin*.

1.3 Out of Scope

Dokumen ini tidak menspesifikasikan logika pembayaran internal SmartBank, mekanisme *routing* JWT API Gateway, algoritma penentuan tarif pengiriman LogistiKita, ataupun manajemen akun dari *service partner* lain.

Tabel 3: Area di Luar dan di Dalam Lingkup SRS

Area	Status Scope	Rasional
Pemrosesan pembayaran	Di luar <i>scope</i>	Seluruh transaksi keuangan diproses SmartBank melalui API Gateway. PasarKita hanya mengirim <i>payment request</i> dan membaca <i>response</i> .
<i>Routing</i> dan JWT validation (inter-service)	Di luar <i>scope</i>	API Gateway (Kelompok 7) bertanggung jawab atas <i>routing</i> , <i>JWT issuance</i> antar-service, dan <i>logging</i> inter-service.
Kalkulasi tarif dan <i>tracking</i> pengiriman internal	Di luar <i>scope</i>	LogistiKita (Kelompok 5) mengelola logika pengiriman. PasarKita hanya men- <i>trigger</i> dan membaca <i>tracking_id</i> .
Manajemen stok <i>supplier</i>	Di luar <i>scope</i>	SupplierHub (Kelompok 4) mengelola <i>supply chain</i> . PasarKita hanya membaca stok lokal dari <i>database</i> sendiri.
Laporan keuangan konsolidasi ekosistem	Di luar <i>scope</i>	UMKM Insight (Kelompok 6) mengonsumsi <i>ledger</i> dari SmartBank. PasarKita hanya menyediakan data transaksi lokal via <i>analytics endpoint</i> .
Manajemen produk CRUD <i>seller</i> (WarungPOS)	Di luar <i>scope</i>	WarungPOS (Kelompok 3) memiliki katalog tersendiri. PasarKita mengelola katalog produk <i>seller</i> yang terdaftar di platformnya sendiri.
Manajemen saldo dompet user	Di dalam <i>scope</i> (partial)	PasarKita menyimpan <i>snapshot</i> saldo lokal dari SmartBank untuk keperluan validasi awal, namun <i>authoritative balance</i> tetap di SmartBank.
Katalog produk, <i>order</i> , dan ulasan	Di dalam <i>scope</i>	PasarKita adalah <i>system of record</i> untuk produk, <i>order</i> , <i>order_items</i> , dan <i>ratings</i> dalam ekosistemnya.

2. Gambaran Produk dan Batas Sistem

2.1 Product Perspective

Aplikasi ini menempati posisi sebagai **Demand Generator B2C** dalam ekosistem *microservices* UMKM RPL 2. Secara teknis, *frontend* berjalan di `pasarkita.vercel.app` (Next.js 16.2) dan *backend* di `pasarkita-api.vercel.app` (Express.js *serverless*), keduanya di-*deploy* terpisah dari repo monorepo yang sama. *Database* menggunakan Supabase PostgreSQL yang diakses eksklusif oleh *backend* Pasarkita.

Semua komunikasi ke *service* eksternal (SmartBank, LogistiKita) melewati API Gateway (Kelompok 7) dengan JWT. Untuk keperluan *development* lokal, tersedia *mock server* di direktori `mock/` yang mensimulasikan SmartBank (:4001) dan LogistiKita (:4002).

2.2 User Classes dan Karakteristik

Sistem melayani beberapa kelas pengguna dengan hak dan ekspektasi yang berbeda. Kebutuhan sistem harus menjaga perbedaan tersebut agar transaksi *marketplace* dapat berjalan tanpa mencampuri domain *service* lain dalam ekosistem.

Tabel 4: User Classes dan Karakteristik

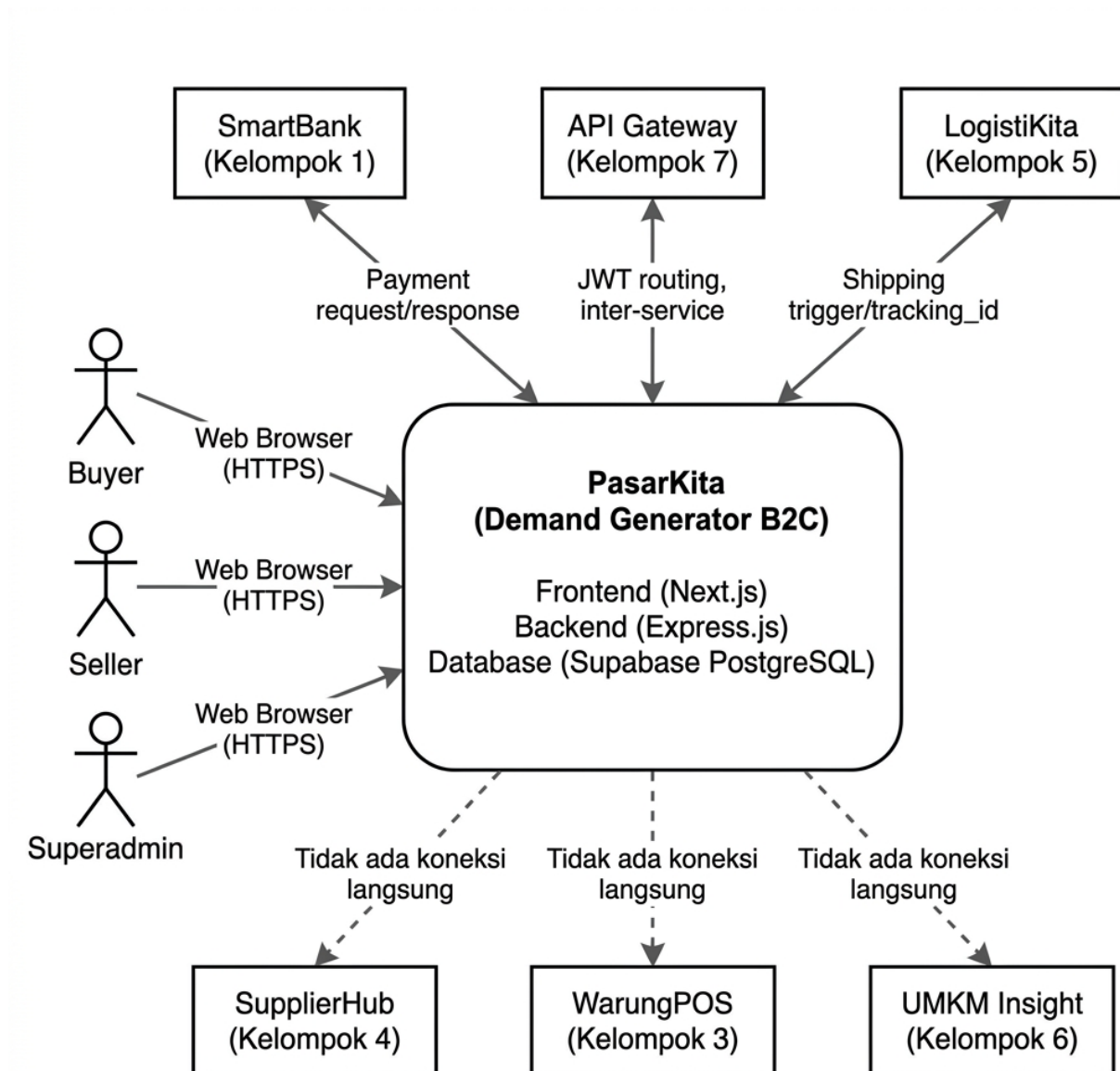
User Class	Tanggung Jawab	Ekspektasi Sistem	Kontrol Akses
Buyer	Menelusuri produk, menambahkan ke keranjang, melakukan <i>checkout</i> , memantau status <i>order</i> , dan memberikan ulasan.	Alur belanja yang mudah, status <i>order real-time</i> , konfirmasi pembayaran jelas, riwayat pesanan tersedia.	JWT role buyer; tidak dapat mengelola produk atau melihat <i>analytics</i> .
Seller	Mengelola katalog produk sendiri (CRUD), memantau <i>order</i> masuk, mengelola stok, dan membalas ulasan.	<i>Dashboard</i> produk yang efisien, notifikasi <i>order</i> baru, visibilitas stok rendah.	JWT role seller; hanya dapat CRUD produk milik sendiri; tidak dapat mengakses data <i>seller</i> lain.
Superadmin	Memantau seluruh operasional platform: manajemen <i>user</i> , <i>order</i> , <i>analytics</i> , moderasi konten, dan <i>audit log</i> .	<i>Dashboard analytics</i> lengkap, kemampuan ban/aktifkan <i>user</i> , <i>update status order</i> manual, akses <i>audit log</i> .	JWT role superadmin; di-buat via <i>insert manual</i> ke Supabase; akses penuh ke semua <i>endpoint</i> admin.
API Gateway	Memvalidasi JWT antar- <i>service</i> , meneruskan <i>payment request</i> ke SmartBank, dan <i>trigger</i> ke LogistiKita.	Kontrak JSON stabil, <i>error response</i> deterministik, <i>idempotency key</i> pada <i>checkout</i> .	<i>Service-to-service</i> via <i>header JWT</i> ekosistem; bukan <i>user</i> biasa.

User Class	Tanggung Jawab	Ekspektasi Sistem	Kontrol Akses
SmartBank	Memproses pembayaran dan mengembalikan <i>transaction_id</i> atau <i>failure response</i> .	Format <i>request</i> konsisten, <i>idempotency</i> terjaga, <i>rollback</i> stok saat pembayaran gagal.	Akses melalui API Gateway; tidak ada akses langsung ke <i>database</i> PasarKita.

2.3 Operating Environment

Tabel 5: Operating Environment Sistem PasarKita

Komponen	Spesifikasi Saat Ini	Implikasi Requirement
Frontend	Next.js 16.2, App Router, TypeScript, Tailwind CSS, shadcn/ui, di-deploy ke Vercel (frontend/).	<i>Routing</i> berbasis App Router; komponen UI harus konsisten dengan shadcn/ui <i>design system</i> .
Backend	Express.js dengan serverless-http, di-deploy ke Vercel <i>serverless function</i> (backend/api/index.js).	Tidak ada <i>persistent in-memory state</i> ; semua <i>state</i> harus tersimpan di <i>database</i> .
Database	Supabase PostgreSQL, diakses via Supabase SDK dan <i>service role key</i> dari <i>backend</i> .	Skema harus menjaga users, products, orders, order_items, ratings, notifications, vouchers, dan audit logs.
State Management	Zustand (auth, cart) dan TanStack Query di <i>frontend</i> .	<i>Cart state</i> bersifat <i>client-side</i> ; sinkronisasi stok terjadi saat <i>checkout</i> di <i>backend</i> .
Auth	JWT yang di-generate <i>backend</i> PasarKita saat <i>login</i> ; validasi JWT antar-service via API Gateway.	Token harus menyertakan <i>role</i> ; <i>middleware</i> auth harus memvalidasi sebelum setiap <i>protected route</i> .
Integrasi Ekosistem	API Gateway di GATEWAY_BASE_URL; SmartBank dan LogistiKita diakses via Gateway.	Koneksi eksternal harus gagal secara terkendali dan tidak merusak data <i>order</i> lokal.
Mock Server (dev)	Express.js lokal di mock/smartbank/:4001 dan mock/logistikita/:4002.	Hanya digunakan saat NODE_ENV=development; <i>production</i> selalu melalui GATEWAY_BASE_URL.
Deploy	Dua project Vercel terpisah dari satu repo monorepo (frontend/ dan backend/).	<i>Environment variables</i> harus dikonfigurasi terpisah per project di Vercel.



Gambar 1: Diagram Konteks Sistem PasarKita dalam Ekosistem UMKM RPL 2

3. Konteks Operasional dan Arsitektur

3.1 Architectural Overview

Aplikasi menggunakan arsitektur **monorepo dua-tier** dengan pemisahan *frontend* dan *backend* yang di-*deploy* secara independen. *Backend* menggunakan pola modular berbasis domain (*src/modules/*) dengan *middleware* terpusat untuk *auth*, *validasi*, dan *error handling*. *Frontend* menggunakan App Router Next.js dengan komponen *shadcn/ui* dan *state management* Zustand.

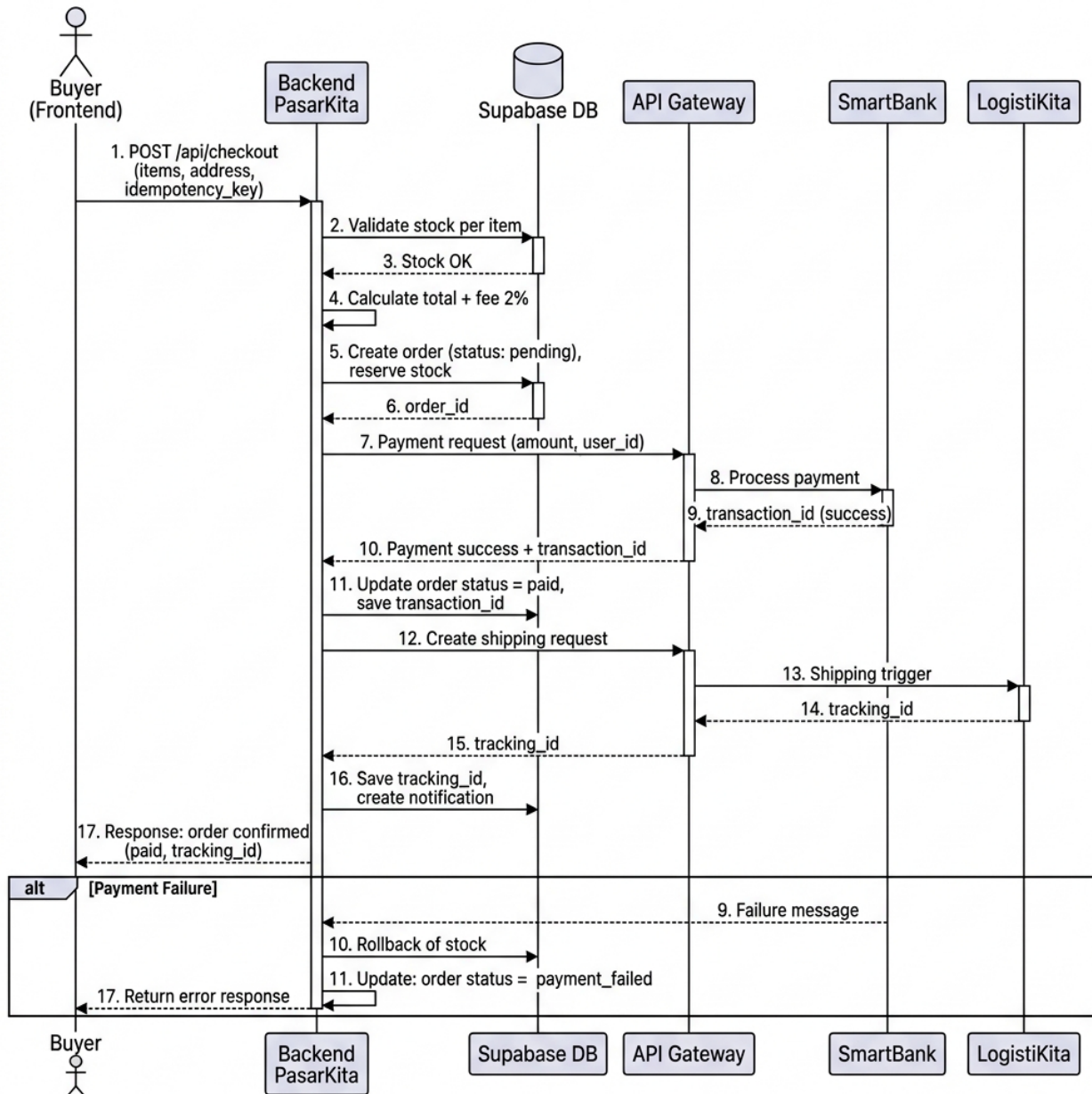
Tabel 6: Lapisan Arsitektur Sistem PasarKita

Lapisan	Artefak Utama	Tanggung Jawab
Frontend: Routing	frontend/app/ (App Router)	Mengarahkan halaman ke komponen React berdasarkan URL.
Frontend: UI Components	frontend/components/	Menyajikan halaman <i>buyer</i> (katalog, <i>cart</i> , <i>checkout</i> , <i>order</i>), <i>seller</i> (<i>dashboard</i> , CRUD produk), dan <i>admin</i> (<i>analytics</i> , manajemen <i>user</i>).
Frontend: State	frontend/store/ (Zustand)	Mengelola <i>auth state</i> (<i>user</i> , <i>token</i>) dan <i>cart state</i> (<i>items</i> , <i>quantity</i>) secara <i>client-side</i> .
Frontend: API Client	frontend/lib/	<i>Wrapper</i> <i>Axios</i> / <i>fetch</i> ke <i>backend</i> API dengan <i>token injection</i> dari Zustand <i>auth store</i> .
Backend: Entry Point	backend/api/index.js	<i>Serverless entry point</i> ; <i>mounting</i> semua <i>router</i> Express.
Backend: Modules	backend/src/modules/ (<i>auth</i> , <i>products</i> , <i>orders</i> , <i>checkout</i> , <i>integrations</i>)	<i>Business logic</i> per domain: autentikasi, katalog produk, manajemen <i>order</i> , alur <i>checkout</i> , dan komunikasi ke <i>service</i> eksternal.
Backend: Middlewares	backend/src/middlewares/ (<i>auth</i> , <i>validate</i> , <i>errorHandler</i>)	Validasi <i>JWT</i> , validasi <i>Zod schema request body</i> , dan <i>centralized error response</i> .
Backend: Integrations	backend/src/integrations/ (<i>smartbank</i> , <i>logistikita</i>)	Adapter HTTP untuk komunikasi ke <i>SmartBank</i> dan <i>LogistiKita</i> via API Gateway.
Backend: Utils	backend/src/utils/ (<i>fee</i> , <i>response</i>)	Kalkulasi <i>fee marketplace</i> (2%), <i>helper</i> format <i>response</i> JSON standar.
Database	Supabase PostgreSQL	Penyimpanan persisten untuk semua entitas domain PasarKita.

3.2 Data Flow

Alur utama dimulai dari registrasi atau *login user* di *frontend*. Setelah autentikasi, *buyer* menelusuri produk, menambahkan ke *cart* (*state* Zustand), dan melakukan *checkout*. Saat *checkout*, *backend* memvalidasi stok, menghitung total + *fee* 2%, membuat *order* dengan status *pending*, lalu mengirim *payment request* ke *SmartBank* via API Gateway.

Jika pembayaran sukses, *backend* memperbarui *order* ke status *paid*, mengirim *trigger* pengiriman ke *LogistiKita*, menerima *tracking_id*, dan mencatat notifikasi. Jika gagal, stok di-*rollback* dan *order* diperbarui ke *payment_failed*. *Superadmin* dapat memantau seluruh alur



Gambar 2: Sequence Diagram Alur Checkout PasarKita

melalui *analytics endpoint* dan *audit log*.

4. Domain Data dan Aturan Bisnis

4.1 Core Domain Entities

Tabel 7: Core Domain Entities PasarKita

Entitas	Deskripsi	Field Penting	Catatan Ownership
users	Akun <i>buyer</i> , <i>seller</i> , dan <i>superadmin</i> .	id, name, email, password_hash, role, is_active, phone, avatar_url	PasarKita adalah <i>system of record</i> untuk akun <i>user</i> dalam ekosistemnya. Saldo dikelola SmartBank.
seller_profiles	Profil toko milik <i>seller</i> .	seller_id, store_name, logo_url, description	Dibuat otomatis saat <i>seller</i> register; dapat diedit oleh <i>seller</i> bersangkutan.
products	Katalog produk yang dijual <i>seller</i> .	id, seller_id, name, description, category, price, stock, is_active, image_url, minimum_stock, is_low_stock	Dikelola <i>seller</i> ; <i>superadmin</i> dapat menonaktifkan.
orders	Transaksi pembelian dari <i>buyer</i> .	id, buyer_id, status, subtotal, fee_marketplace, total, shipping_address, transaction_id, tracking_id, idempotency_key	Dibuat saat <i>checkout</i> ; status diperbarui berdasarkan respons SmartBank dan LogistiKita.
order_items	Detail item dalam satu <i>order</i> .	order_id, product_id, qty, price_at_purchase, product_name_at_purchase, product_discount_per_unit	<i>Snapshot</i> harga saat transaksi; tidak berubah meski harga produk berubah setelah <i>order</i> .
order_status_history	Riwayat perubahan status <i>order</i> .	order_id, status, actor_id, source, note, created_at	Digunakan untuk <i>audit trail</i> setiap perubahan status <i>order</i> .
ratings	Ulasan <i>buyer</i> terhadap produk setelah <i>order</i> selesai.	order_id, product_id, buyer_id, rating, comment, image_urls, seller_reply	Hanya dapat dibuat oleh <i>buyer</i> yang memiliki <i>order</i> delivered untuk produk bersangkutan.
notifications	Notifikasi <i>in-app</i> untuk semua <i>role</i> .	user_id, order_id, type, title, message, href, read_at	Di-generate <i>backend</i> saat <i>event order</i> (paid, shipped, delivered, rating).
vouchers	Kode diskon untuk <i>buyer</i> .	code, discount_type, discount_value, min_purchase, max_uses, valid_until	Dikelola <i>superadmin</i> ; dapat berupa persentase atau nominal tetap.
product_discounts	Diskon langsung pada produk tertentu.	product_id, discount_type, discount_value, valid_from, valid_until	Dikelola <i>seller</i> ; diperhitungkan saat kalkulasi subtotal.

Entitas	Deskripsi	Field Penting	Catatan Ownership
admin_audit_logs	Catatan aksi administratif <i>superadmin</i> .	actor_id, action, target_type, target_id, reason, before_data, after_data	Di-generate otomatis setiap aksi sensitif <i>superadmin</i> (ban user, update status, dsb.).
integration_logs	Log komunikasi ke <i>service</i> eksternal.	service, operation, success, duration_ms, order_id, status_code, error_code	Digunakan untuk <i>monitoring</i> kesehatan integrasi SmartBank dan LogistiKita.

4.2 Business Rules

Tabel 8: Business Rules Sistem PasarKita

ID	Aturan Bisnis	Dampak Implementasi
BR-01	<i>Fee marketplace</i> adalah 2% dari subtotal, dibebankan ke <i>buyer</i> pada setiap transaksi.	backend/src/utils/fee.js menghitung <code>fee_marketplace = Math.ceil(subtotal * 0.02)</code> ; <i>frontend</i> menampilkan rincian <i>fee</i> sebelum konfirmasi.
BR-02	Saldo awal setiap <i>user</i> baru yang didaftarkan di SmartBank adalah Rp 50.000.	<i>Backend</i> harus memanggil SmartBank untuk inisiasi saldo saat registrasi berhasil; <i>failure handling</i> jika SmartBank tidak tersedia.
BR-03	Maksimum transaksi adalah 10x per <i>user</i> per hari dengan <i>cooldown</i> 10–30 detik antar transaksi.	<i>Backend</i> memvalidasi frekuensi transaksi via <i>query orders</i> sebelum proses <i>checkout</i> ; <i>rate limiting</i> diterapkan di <i>middleware</i> .
BR-04	Stok harus divalidasi dan di-reserve sebelum <i>payment request</i> dikirim ke SmartBank.	<i>Checkout module</i> memvalidasi stok per item, men-set <code>stock_reserved = true</code> pada <i>order</i> pending, dan me-rollback stok jika pembayaran gagal.
BR-05	Semua komunikasi ke SmartBank dan LogistiKita pada <i>environment production</i> harus melalui <code>GATEWAY_BASE_URL</code> .	<i>Integrations module</i> membaca <code>NODE_ENV</code> ; jika <i>development</i> menggunakan <i>mock URL</i> , jika <i>production</i> menggunakan <code>GATEWAY_BASE_URL</code> .
BR-06	<i>Idempotency key</i> wajib digunakan pada setiap <i>checkout request</i> untuk mencegah <i>double order</i> .	<i>Backend</i> menghasilkan dan menyimpan <i>idempotency_key</i> (UUID) per <i>checkout</i> ; <i>request</i> duplikat dengan <i>key</i> yang sama mengembalikan <i>order</i> yang sudah ada.
BR-07	<i>Buyer</i> hanya dapat memberikan <i>rating</i> setelah <i>order</i> memiliki status <i>delivered</i> .	<i>Ratings module</i> memvalidasi status <i>order</i> sebelum menerima <i>rating submission</i> ; satu <i>order</i> satu <i>rating</i> per produk.
BR-08	<i>Seller</i> hanya dapat mengelola (CRUD) produk miliknya sendiri.	<i>Products module</i> memvalidasi <i>seller_id</i> dari JWT sama dengan <i>seller_id</i> pada produk sebelum izinkan <i>update/delete</i> .
BR-09	<i>Superadmin</i> tidak dapat mendaftarkan diri sendiri via API registrasi publik.	<i>Role</i> <i>superadmin</i> hanya dapat dibuat via <i>insert</i> manual ke Supabase; <i>endpoint</i> <code>/api/auth/register</code> hanya menerima <i>role</i> <i>buyer</i> atau <i>seller</i> .

ID	Aturan Bisnis	Dampak Implementasi
BR-10	Harga <code>price_at_purchase</code> pada <code>order_items</code> adalah <i>snapshot</i> harga saat transaksi dan tidak boleh berubah.	Saat <i>order</i> dibuat, harga produk disalin ke <code>price_at_purchase</code> ; perubahan harga produk setelahnya tidak memengaruhi <i>order</i> yang sudah ada.

5. Kebutuhan Fungsional

Kebutuhan fungsional ditulis dengan bentuk *shall statement* agar dapat menjadi dasar implementasi dan pengujian. Prioritas **Must** menunjukkan kemampuan inti rilis operasional; **Should** menunjukkan kemampuan penting yang memperkuat operasi; **Could** menunjukkan peningkatan lanjutan.

Tabel 9: Kebutuhan Fungsional Sistem Pasarkita

ID	Area	Requirement	Priority	Acceptance Criteria
FR-01	Auth: Register	Sistem <i>shall</i> menyediakan <i>endpoint</i> registrasi <i>user</i> dengan <i>role</i> <i>buyer</i> atau <i>seller</i> , memvalidasi <i>input</i> dengan <i>Zod</i> , dan menyimpan <i>password</i> dalam bentuk <i>hash</i> .	Must	<i>User</i> baru tersimpan di <i>users</i> dengan <i>password_hash</i> ; <i>role</i> selain <i>buyer/seller</i> ditolak dengan pesan eksplisit.
FR-02	Auth: Login	Sistem <i>shall</i> menyediakan <i>endpoint login</i> yang memverifikasi kredensial dan mengembalikan <i>JWT</i> berisi <i>user_id</i> , <i>role</i> , dan <i>exp</i> .	Must	<i>Login</i> dengan kredensial valid mengembalikan <i>JWT</i> ; kredensial salah mengembalikan 401 tanpa membocorkan detail <i>database</i> .
FR-03	Seller Profile	Sistem <i>shall</i> membuat <i>seller_profile</i> secara otomatis saat <i>seller</i> berhasil registrasi, dan memungkinkan <i>seller</i> memperbarui <i>store_name</i> , <i>logo_url</i> , dan <i>description</i> .	Must	<i>seller_profiles</i> terisi setelah registrasi <i>seller</i> ; <i>update</i> profil tersimpan dan terbaca di halaman toko.
FR-04	Catalog: Browse	Sistem <i>shall</i> menyediakan <i>endpoint GET /api/products</i> dengan filter kategori, <i>search</i> nama, dan <i>pagination</i> untuk semua <i>role</i> termasuk <i>guest</i> .	Must	<i>Response</i> mengembalikan <i>array</i> produk aktif dengan <i>field</i> lengkap; filter dan <i>pagination</i> berfungsi sesuai parameter <i>query</i> .
FR-05	Catalog: Detail	Sistem <i>shall</i> menyediakan <i>endpoint GET /api/products/:id</i> yang mengembalikan detail produk termasuk informasi <i>seller</i> dan rata-rata <i>rating</i> .	Must	<i>Response</i> berisi data produk, <i>store_name seller</i> , <i>avg_rating</i> , dan <i>total_reviews</i> .
FR-06	Product: CRUD	Sistem <i>shall</i> memungkinkan <i>seller</i> membuat, memperbarui, dan menghapus (<i>soft delete</i> via <i>is_active = false</i>) produk miliknya sendiri, dengan validasi <i>price > 0</i> dan <i>stock >= 0</i> .	Must	<i>Seller</i> hanya dapat <i>CRUD</i> produk dengan <i>seller_id</i> miliknya; akses ke produk <i>seller</i> lain menghasilkan 403.
FR-07	Product Discount	Sistem <i>should</i> memungkinkan <i>seller</i> mengatur diskon langsung pada produk (<i>product_discounts</i>) dengan periode validitas dan tipe (<i>persentase/nominal</i>).	Should	Diskon aktif diperhitungkan dalam kalkulasi subtotal saat <i>checkout</i> ; diskon kedaluwarsa diabaikan.

ID	Area	Requirement	Priority	Acceptance Criteria
FR-08	Cart	Sistem <i>shall</i> mengelola keranjang belanja di sisi <i>frontend</i> (Zustand), termasuk tambah item, ubah kuantitas, dan hapus item, dengan validasi stok saat <i>checkout</i> .	Must	<i>Cart state</i> tersimpan di Zustand; stok divalidasi ulang di <i>backend</i> saat <i>checkout request</i> dikirim.
FR-09	Checkout & Payment	Sistem <i>shall</i> memproses <i>checkout</i> dengan langkah: validasi stok → kalkulasi total + <i>fee</i> 2% → buat <i>order pending</i> → kirim <i>payment request</i> ke SmartBank via Gateway → <i>update status order</i> berdasarkan <i>response</i> .	Must	<i>Order paid</i> muncul setelah SmartBank sukses; <i>order payment_failed</i> dan stok di- <i>rollback</i> jika SmartBank gagal.
FR-10	Idempotency Checkout	Sistem <i>shall</i> menerapkan <i>idempotency key</i> pada setiap <i>checkout</i> untuk mencegah <i>double order</i> akibat <i>retry request</i> .	Must	<i>Request checkout</i> duplikat dengan <i>key</i> yang sama mengembalikan <i>order</i> yang sudah ada tanpa membuat <i>order</i> baru.
FR-11	Shipping Trigger	Sistem <i>shall</i> mengirimkan <i>trigger</i> pengiriman ke LogistiKita via API Gateway setelah <i>order</i> berhasil dibayar, dan menyimpan <i>tracking_id</i> yang diterima.	Must	<i>Order paid</i> memiliki <i>tracking_id</i> terisi; kegagalan <i>trigger</i> pengiriman dicatat di <i>integration_logs</i> dan tidak membatalkan <i>order</i> .
FR-12	Fee Simulation	Sistem <i>should</i> menyediakan <i>endpoint</i> POST <i>/api/fee/calculate</i> untuk simulasi kalkulasi <i>fee</i> tanpa melakukan transaksi nyata.	Should	<i>Response</i> mengembalikan <i>subtotal</i> , <i>fee_marketplace</i> , <i>fee_discount</i> , <i>voucher_discount</i> , dan <i>total</i> berdasarkan <i>input</i> .
FR-13	Voucher	Sistem <i>should</i> memungkinkan <i>buyer</i> menggunakan kode <i>voucher</i> saat <i>checkout</i> dengan validasi <i>min_purchase</i> , <i>max_uses</i> , dan <i>valid_until</i> .	Should	Diskon <i>voucher</i> diperhitungkan pada <i>field</i> <i>voucher_discount</i> di <i>order</i> ; <i>voucher</i> habis/kedaluwarsa ditolak dengan pesan eksplisit.
FR-14	Order: Buyer	Sistem <i>shall</i> menyediakan <i>endpoint</i> GET <i>/api/orders</i> dan GET <i>/api/orders/:id</i> bagi <i>buyer</i> untuk melihat daftar dan detail <i>order</i> miliknya.	Must	<i>Buyer</i> hanya melihat <i>order</i> milik sendiri; akses ke <i>order buyer</i> lain menghasilkan 403.
FR-15	Order: Seller	Sistem <i>shall</i> menyediakan akses <i>seller</i> ke <i>order</i> yang mengandung produk miliknya, termasuk status dan informasi pengiriman.	Must	<i>Seller</i> dapat melihat <i>order</i> masuk yang berisi produknya; informasi <i>buyer</i> ditampilkan sesuai kebutuhan <i>fulfillment</i> .
FR-16	Order Status Update	Sistem <i>shall</i> memungkinkan <i>superadmin</i> memperbarui status <i>order</i> secara manual dan mencatat perubahan ke <i>order_status_history</i> .	Must	Perubahan status tersimpan di <i>orders</i> dan <i>order_status_history</i> dengan <i>actor_id</i> <i>superadmin</i> dan <i>timestamp</i> .

ID	Area	Requirement	Priority	Acceptance Criteria
FR-17	Ratings & Reviews	Sistem <i>shall</i> memungkinkan <i>buyer</i> memberikan <i>rating</i> (1–5) dan komentar untuk produk yang sudah diterima (<i>status</i> = <i>delivered</i>), dan memungkinkan <i>seller</i> membalas ulasan.	Must	<i>Rating</i> tersimpan setelah validasi status <i>order</i> ; <i>seller_reply</i> dapat diperbarui <i>seller</i> ; rata-rata <i>rating</i> terhitung di <i>endpoint</i> produk.
FR-18	Notifications	Sistem <i>should</i> men- <i>generate</i> notifikasi <i>in-app</i> untuk <i>event order</i> (<i>paid</i> , <i>shipped</i> , <i>delivered</i>) dan <i>rating</i> , dapat ditandai sebagai sudah dibaca.	Should	Notifikasi muncul di daftar <i>user</i> yang relevan; <i>endpoint</i> PATCH menandai <i>read_at</i> ; notifikasi tidak dibaca ditampilkan sebagai <i>badge</i> .
FR-19	Admin: User	Sistem <i>shall</i> menyediakan <i>superadmin</i> kemampuan untuk melihat semua <i>user</i> , ban (<i>set is_active</i> = <i>false</i>), dan mengaktifkan kembali <i>user</i> dengan alasan yang tercatat di <i>admin_audit_logs</i> .	Must	Status <i>user</i> berubah setelah aksi admin; aksi tercatat di <i>audit log</i> dengan <i>before_data</i> dan <i>after_data</i> .
FR-20	Admin: Analytics	Sistem <i>shall</i> menyediakan <i>endpoint</i> GET <i>/api/admin/analytics</i> yang mengembalikan metrik platform: total <i>revenue</i> , total <i>order</i> , distribusi status <i>order</i> , produk terlaris, dan <i>revenue</i> per <i>seller</i> .	Must	<i>Response</i> mengembalikan data agregat akurat; <i>dashboard</i> admin menampilkan visualisasi metrik tanpa <i>query</i> manual.
FR-21	Integration Health Check	Sistem <i>should</i> menyediakan <i>endpoint</i> internal admin untuk memeriksa status koneksi ke API Gateway, SmartBank, dan LogistiKita.	Should	<i>Endpoint</i> mengembalikan status <i>online/offline</i> , <i>http_code</i> , <i>response_ms</i> , dan <i>last_checked_at</i> per <i>service</i> .
FR-22	Low Stock Alert	Sistem <i>should</i> menampilkan indikator stok rendah pada produk yang memiliki <i>stock</i> <= <i>minimum_stock</i> di <i>dashboard seller</i> .	Should	<i>is_low_stock</i> ter- <i>update</i> secara otomatis; <i>dashboard seller</i> menampilkan <i>badge</i> peringatan pada produk yang relevan.

6. Kebutuhan Non-Fungsional

Kebutuhan non-fungsional menentukan kualitas sistem yang harus dipenuhi di luar perilaku fitur langsung. Kategori ini penting karena sistem memproses transaksi keuangan nyata dalam ekosistem *microservices* multi-kelompok.

Tabel 10: Kebutuhan Non-Fungsional Sistem PasarKita

ID	Quality Attribute	Requirement	Verification
NFR-01	Security: Password	Sistem <i>shall</i> menyimpan <i>password</i> dalam bentuk <i>hash</i> (bcrypt) dan tidak pernah mengembalikan <i>password_hash</i> dalam <i>response</i> API.	Audit <i>endpoint user</i> ; tidak ada <i>password_hash</i> di <i>response</i> .
NFR-02	Security: JWT	Sistem <i>shall</i> memvalidasi JWT pada setiap <i>protected route</i> sebelum memproses <i>request</i> ; token kedaluwarsa atau tidak valid mengembalikan 401.	<i>Request</i> tanpa token atau dengan token tidak valid ditolak secara konsisten.
NFR-03	Access Control	<i>Buyer</i> hanya mengakses <i>resource</i> miliknya; <i>seller</i> hanya mengelola produk miliknya; <i>superadmin</i> memiliki akses penuh; <i>endpoint admin</i> dilindungi <i>middleware role check</i> .	<i>Cross-role access</i> menghasilkan 403; <i>test</i> dengan <i>token</i> berbeda <i>role</i> .
NFR-04	Data Integrity: Checkout	Sistem <i>shall</i> memastikan <i>atomicity</i> pada proses <i>checkout</i> : stok tidak berkurang jika <i>payment</i> gagal; <i>order</i> tidak dibuat jika validasi stok gagal.	Simulasi <i>payment failure</i> ; verifikasi stok tidak berubah dan <i>order</i> tidak terbuat.
NFR-05	Interoperability	Kontrak JSON API PasarKita <i>shall</i> mempertahankan <i>backward compatibility</i> ; <i>field</i> yang dikonsumsi <i>service</i> lain tidak boleh dihapus atau diubah tipenya tanpa koordinasi.	<i>Service partner</i> masih dapat membaca <i>response</i> API setelah perubahan; <i>field</i> baru bersifat <i>additive</i> .
NFR-06	Availability	Sistem <i>shall</i> dapat di-deploy dan berjalan di Vercel (<i>frontend</i> + <i>backend</i> project terpisah) dengan <i>environment variables</i> yang terdokumentasi.	<i>vercel.json</i> mengonfigurasi <i>routing</i> dengan benar; kedua <i>project</i> dapat di-deploy dari repo yang sama.
NFR-07	Failure Handling	Kegagalan koneksi ke API Gateway, SmartBank, atau LogistiKita <i>shall</i> menghasilkan <i>error response</i> yang terkendali dan dicatat di <i>integration_logs</i> ; tidak menghentikan seluruh sistem.	Matikan <i>mock service</i> ; verifikasi <i>error response</i> bermakna dan log tercatat.
NFR-08	Maintainability	Kode <i>backend</i> <i>shall</i> mempertahankan pemisahan modular berbasis domain sehingga <i>business logic</i> , integrasi eksternal, dan <i>middleware</i> tidak bercampur.	Modul domain tetap berada di <i>src/modules/</i> ; integrasi di <i>src/integrations/</i> ; <i>middleware</i> di <i>src/middlewares/</i> .
NFR-09	Auditability	Setiap aksi sensitif <i>superadmin</i> <i>shall</i> menulis <i>admin_audit_logs</i> dengan <i>before_data</i> dan <i>after_data</i> untuk keperluan <i>audit trail</i> .	Aksi ban <i>user</i> dan <i>update status order</i> menghasilkan entri <i>audit log</i> yang dapat ditelusuri.

ID	Quality Attribute	Requirement	Verification
NFR-10	Usability	Alur <i>checkout shall</i> menampilkan rincian harga (subtotal, <i>fee</i> , diskon, total) secara eksplisit sebelum konfirmasi pembayaran.	<i>Fee breakdown</i> terlihat di halaman <i>review order</i> ; tidak ada biaya tersembunyi.
NFR-11	Performance	<i>Endpoint</i> katalog produk <i>should</i> mengembalikan <i>response</i> di bawah 2 detik untuk halaman pertama dengan 50 item tanpa filter berat.	<i>Load test</i> dengan 50 produk; <i>response time</i> diukur dari <i>client</i> .
NFR-12	Privacy	Data sensitif <i>user</i> (<i>email</i> , <i>phone</i> , <i>password_hash</i>) <i>shall</i> tidak ditampilkan kepada <i>user</i> lain kecuali <i>superadmin</i> pada konteks manajemen <i>user</i> yang sah.	<i>Endpoint</i> produk dan <i>order</i> tidak mengembalikan data sensitif <i>seller/buyer</i> secara berlebihan.

7. Antarmuka Eksternal

7.1 User Interface Requirements

UI harus mendukung pekerjaan operasional yang berbeda per *role*: *buyer* menelusuri dan membeli produk, *seller* mengelola katalog dan *order* masuk, *superadmin* memantau platform secara keseluruhan. Karena itu halaman tidak hanya berfungsi sebagai *form*, tetapi juga sebagai *control surface* untuk status transaksi, stok, dan kesehatan integrasi.

Tabel 11: User Interface Requirements per Route

UI / Route	Primary User	Requirement
/	Semua / Guest	<i>Landing page</i> harus menampilkan katalog produk unggulan, <i>banner</i> promo, dan akses <i>login/register</i> .
/auth/login dan /auth/register	Semua	Halaman <i>auth</i> harus jelas membedakan opsi <i>register</i> sebagai <i>buyer</i> atau <i>seller</i> ; tidak ada opsi <i>superadmin</i> .
/products	Buyer / Guest	Halaman katalog harus mendukung filter kategori, <i>search</i> , <i>sorting</i> harga, dan <i>pagination</i> .
/products/:id	Buyer / Guest	Halaman detail produk harus menampilkan gambar, deskripsi, harga, stok, info toko, dan daftar ulasan.
/cart	Buyer	Halaman keranjang harus menampilkan item, kuantitas (<i>editable</i>), subtotal per item, dan tombol <i>checkout</i> .
/checkout	Buyer	Halaman <i>checkout</i> harus menampilkan ringkasan <i>order</i> , kolom alamat pengiriman, <i>input voucher</i> , <i>fee breakdown</i> lengkap, dan tombol konfirmasi bayar.
/orders dan /orders/:id	Buyer	Halaman <i>order</i> harus menampilkan status terkini, <i>tracking info</i> , dan opsi <i>review</i> setelah <i>delivered</i> .
/seller/dashboard	Seller	<i>Dashboard seller</i> harus menampilkan ringkasan produk aktif, stok rendah, dan <i>order</i> masuk terbaru.
/seller/products	Seller	Halaman manajemen produk harus mendukung CRUD produk dengan form validasi dan <i>upload</i> gambar.
/seller/orders	Seller	Halaman <i>order seller</i> menampilkan <i>order</i> masuk dengan filter status dan detail item per <i>order</i> .
/admin	Superadmin	<i>Dashboard admin</i> harus menampilkan metrik platform: total <i>revenue</i> , <i>order count</i> , <i>user</i> aktif, dan grafik tren.
/admin/users	Superadmin	Halaman manajemen <i>user</i> menampilkan semua <i>user</i> dengan aksi <i>ban/aktifkan</i> dan <i>input</i> alasan.
/admin/orders	Superadmin	Halaman <i>order admin</i> menampilkan semua <i>order</i> dengan kemampuan filter status dan <i>update</i> manual.
/admin/analytics	Superadmin	Halaman <i>analytics</i> menampilkan produk terlaris, <i>revenue</i> per <i>seller</i> , distribusi kategori, dan tren transaksi.

7.2 API Requirements

Tabel 12: Daftar API Endpoint PasarKita

Endpoint	Method	Auth	Deskripsi
/api/auth/register	POST	—	Registrasi user baru; <i>role</i> buyer atau seller.
/api/auth/login	POST	—	Login; mengembalikan JWT.
/api/products	GET	—	Browse semua produk aktif; mendukung filter dan <i>pagination</i> .
/api/products/:id	GET	—	Detail produk termasuk <i>rating</i> agregat.
/api/products	POST	<i>seller</i>	Tambah produk baru milik <i>seller</i> .
/api/products/:id	PUT	<i>seller</i> / <i>superadmin</i>	Edit produk; <i>seller</i> hanya produk milik sendiri.
/api/products/:id	DELETE	<i>seller</i> / <i>superadmin</i>	Soft delete produk (<i>is_active</i> = false).
/api/checkout	POST	<i>buyer</i>	Proses <i>checkout</i> lengkap termasuk <i>payment request</i> ke SmartBank.
/api/fee/calculate	POST	—	Simulasi kalkulasi <i>fee</i> dan diskon tanpa transaksi.
/api/orders	GET	<i>buyer</i> / <i>seller</i> / <i>superadmin</i>	Daftar <i>order</i> ; difilter berdasarkan <i>role</i> yang <i>request</i> .
/api/orders/:id	GET	<i>buyer</i> / <i>seller</i> / <i>superadmin</i>	Detail <i>order</i> termasuk <i>items</i> dan status <i>history</i> .
/api/orders/:id/status	PATCH	<i>superadmin</i>	Update status <i>order</i> manual.
/api/orders/:id/ratings	POST	<i>buyer</i>	Submit <i>rating</i> setelah <i>order</i> delivered.
/api/admin/users	GET	<i>superadmin</i>	Semua <i>user</i> dengan filter dan <i>pagination</i> .
/api/admin/users/:id/status	PATCH	<i>superadmin</i>	Ban atau aktifkan <i>user</i> .
/api/admin/analytics	GET	<i>superadmin</i>	Metrik platform: <i>revenue</i> , <i>order</i> , produk terlaris.
/api/seller/profile	GET / PUT	<i>seller</i>	Baca dan <i>update</i> profil toko <i>seller</i> .
/api/notifications	GET	semua <i>role</i>	Daftar notifikasi milik <i>user</i> yang sedang <i>login</i> .
/api/notifications/:id/read	PATCH	semua <i>role</i>	Tandai notifikasi sudah dibaca.

7.3 Data Exchange Contract

Kontrak JSON pada *checkout request* dan *response* harus diperlakukan sebagai *integration contract* dengan API Gateway dan *service partner*. Perubahan nama *field* atau tipe data dapat memutus integrasi dengan SmartBank atau LogistiKita.

```
// POST /api/checkout -- Request Body
{
  "items": [
    { "product_id": "uuid", "qty": 2 }
  ]
}
```

```
] ,  
  "shipping_address": "Jl. Contoh No. 1, Bandung",  
  "voucher_code": "UMKM2026",  
  "idempotency_key": "uuid-v4"  
}
```

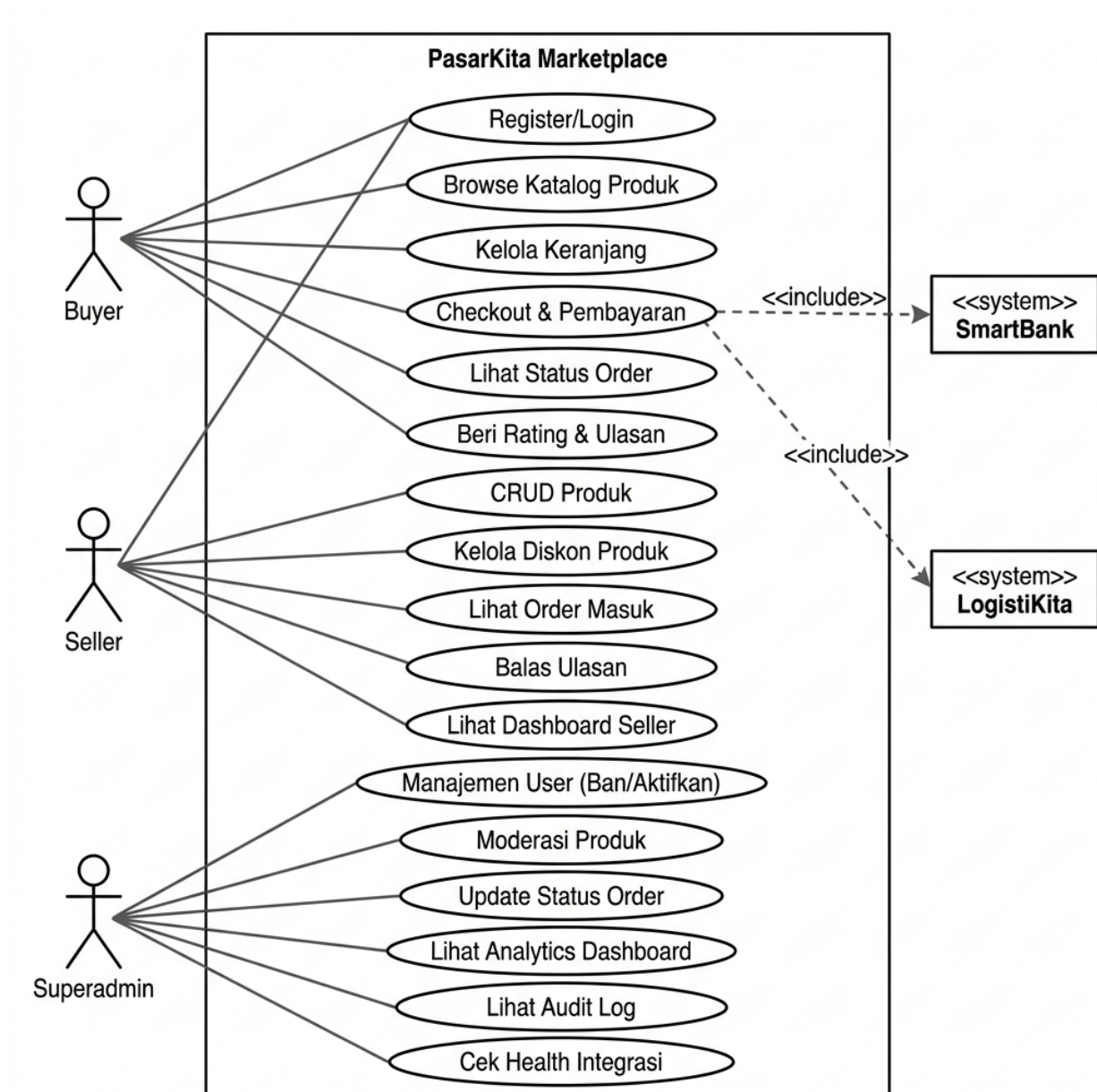
Listing 1: POST /api/checkout: Request Body

```
// POST /api/checkout -- Response (sukses)  
{  
  "order_id": "uuid",  
  "status": "paid",  
  "subtotal": 100000,  
  "fee_marketplace": 2000,  
  "voucher_discount": 5000,  
  "total": 97000,  
  "transaction_id": "SB-TXN-XXXX",  
  "tracking_id": "LK-TRACK-XXXX"  
}
```

Listing 2: POST /api/checkout: Response (sukses)

8. Workflow Operasional

Workflow berikut menggambarkan operasi sistem pada kondisi penggunaan nyata. Setiap workflow dapat dijadikan dasar *test case end-to-end* dan *smoke test* setelah perubahan fitur.



Gambar 3: Use Case Diagram Utama Sistem PasarKita

Tabel 13: Workflow Operasional Utama Sistem PasarKita

Workflow	Trigger	Main Success Scenario	Failure / Control
User Registration	User membuka /auth/register.	Sistem memvalidasi <i>input</i> , <i>hash password</i> , simpan <i>user</i> , inisiasi saldo Rp 50.000 ke SmartBank, kembalikan JWT.	Jika SmartBank tidak tersedia, <i>user</i> tersimpan namun saldo belum terinisiasi; perlu <i>retry</i> manual atau <i>background job</i> .
Product Listing	<i>Seller</i> membuka /seller/products.	<i>Seller</i> membuat produk dengan <i>form valid</i> ; produk aktif dan tampil di katalog <i>buyer</i> .	Validasi Zod gagal mengembalikan daftar <i>field error</i> eksplisit; produk tidak tersimpan.
Checkout & Payment	<i>Buyer</i> menekan tombol “Bayar” di halaman <i>checkout</i> .	<i>Backend</i> validasi stok → kalkulasi <i>fee</i> → buat <i>order pending</i> → <i>payment request</i> ke SmartBank → <i>update paid</i> → <i>trigger</i> LogistiKita → kembalikan konfirmasi.	Stok tidak cukup: 400 INSUFFICIENT_STOCK. SmartBank gagal: 402 PAYMENT_FAILED, stok di-rollback, <i>order payment_failed</i> .
Order Tracking	<i>Buyer</i> membuka /orders/:id.	Sistem menampilkan status <i>order</i> terkini, <i>tracking_id</i> , estimasi pengiriman, dan riwayat status.	Jika <i>tracking_id</i> belum ada (<i>shipping trigger</i> gagal), tampilkan status “Menunggu Pengiriman” dengan notifikasi admin.
Review Submission	<i>Buyer</i> membuka detail <i>order</i> dengan status <i>delivered</i> .	<i>Buyer</i> mengisi <i>rating</i> dan komentar; sistem memvalidasi status <i>order</i> ; <i>rating</i> tersimpan; rata-rata <i>rating</i> produk diperbarui.	<i>Rating</i> ditolak jika <i>order</i> belum <i>delivered</i> atau sudah pernah di- <i>rating</i> ; pesan <i>error</i> eksplisit ditampilkan.
Admin User Ban	<i>Superadmin</i> membuka /admin/users.	<i>Superadmin</i> memasukkan alasan ban; sistem men-set <i>is_active</i> = false, menulis <i>admin_audit_logs</i> , dan mengembalikan status terbaru.	<i>User</i> yang di-ban tidak dapat <i>login</i> ; token aktif <i>user</i> tersebut diinvalidasi pada <i>request</i> berikutnya.
Integration Health Check	<i>Superadmin</i> membuka halaman status integrasi.	Sistem melakukan <i>ping</i> ke API Gateway, SmartBank, dan LogistiKita; menampilkan status online/offline dan <i>response_ms</i> .	Jika <i>service</i> eksternal <i>offline</i> , <i>dashboard</i> tetap berjalan; status ditampilkan sebagai <i>offline</i> dengan pesan <i>error</i> terakhir.
Analytics Pull	<i>Superadmin</i> membuka /admin/analytics.	Sistem mengembalikan metrik agregat: total <i>revenue</i> , <i>order count</i> , distribusi status, produk terlaris, dan <i>revenue per seller</i> .	Jika <i>query timeout</i> , <i>dashboard</i> menampilkan pesan <i>error parsial</i> ; data lain yang berhasil dimuat tetap ditampilkan.

9. Risiko, Kontrol, dan Acceptance Criteria

Karena sistem berada di antara proses transaksi keuangan dan logistik dalam ekosistem *microservices*, risiko utama bukan hanya *bug* UI, tetapi juga inkonsistensi stok, *double charge*, dan perubahan kontrak API yang tidak kompatibel.

Tabel 14: Risiko, Kontrol, dan Acceptance Criteria

Risk ID	Risiko	Kontrol Wajib	Acceptance Criteria
R-01	<i>Double order</i> akibat <i>retry checkout</i> .	<i>Idempotency key</i> wajib pada setiap <i>checkout</i> ; <i>backend</i> mendeteksi dan mengembalikan <i>order existing</i> .	<i>Checkout</i> duplikat dengan <i>key</i> yang sama tidak membuat <i>order</i> baru; <i>response</i> mengembalikan <i>order</i> pertama.
R-02	Stok berkurang meski pembayaran gagal.	Stok di-reserve saat <i>order pending</i> ; <i>rollback</i> stok dilakukan segera setelah SmartBank mengembalikan <i>failure</i> .	Setelah <i>payment failure</i> , stock produk kembali ke nilai sebelum <i>checkout</i> ; <i>order</i> berstatus <i>payment_failed</i> .
R-03	Service SmartBank atau LogistiKita tidak tersedia saat <i>checkout</i> .	<i>Failure handling</i> dengan <i>timeout</i> , <i>error response</i> bermakna, pencatatan di <i>integration_logs</i> ; <i>checkout</i> tidak menggantung tanpa batas.	<i>Timeout</i> ke SmartBank menghasilkan 503 SERVICE_UNAVAILABLE dengan pesan yang dapat dimengerti <i>buyer</i> ; <i>order</i> tidak terbuat.
R-04	Perubahan kontrak API memutus integrasi dengan <i>service partner</i> .	<i>Backward compatibility</i> wajib; <i>field</i> baru bersifat <i>additive</i> ; perubahan <i>breaking</i> didiskusikan lintas kelompok sebelum <i>deploy</i> .	<i>Service partner</i> dapat membaca <i>response</i> API setelah <i>update</i> tanpa perubahan kode di sisi mereka untuk <i>field</i> yang sudah ada.
R-05	Akses tidak sah ke data <i>user</i> atau <i>order</i> lain.	<i>Role-based middleware</i> memvalidasi kepemilikan <i>resource</i> ; <i>buyer</i> hanya melihat <i>order</i> sendiri; <i>seller</i> hanya kelola produk sendiri.	<i>Request</i> akses ke <i>resource user</i> lain menghasilkan 403 <i>Forbidden</i> ; tidak ada data <i>user</i> lain yang ter-expose.
R-06	Harga produk berubah setelah <i>order</i> dibuat, memengaruhi laporan.	<i>price_at_purchase</i> adalah <i>snapshot immutable</i> saat transaksi; kalkulasi <i>revenue</i> menggunakan <i>field</i> ini.	<i>Update</i> harga produk tidak memengaruhi nilai <i>subtotal</i> pada <i>order</i> yang sudah ada.
R-07	<i>Superadmin</i> tidak dapat diaudit.	<i>admin_audit_logs</i> menulis setiap aksi sensitif dengan <i>before_data</i> dan <i>after_data</i> .	Setiap aksi ban <i>user</i> dan <i>update</i> status <i>order</i> menghasilkan entri <i>audit log</i> yang dapat ditelusuri oleh aktor.

10. Matriks Ketertelusuran

Traceability matrix menghubungkan *objective* produk dengan *requirement* dan bukti verifikasi. Matriks ini membantu *developer* dan QA memastikan perubahan kode tetap menjaga tujuan sistem.

Tabel 15: Matriks Ketertelusuran Requirement PasarKita

Objective	Requirement IDs	Evidence / Verification
User dapat mendaftar dan login dengan role yang tepat.	FR-01, FR-02, FR-03, NFR-01, NFR-02	Test register <i>buyer/seller/superadmin</i> ; test login valid/invalid; test token <i>expired</i> .
Buyer dapat menelusuri dan membeli produk UMKM.	FR-04, FR-05, FR-08, FR-09, FR-10, NFR-04	Test katalog dengan filter; test <i>checkout</i> sukses; test <i>checkout</i> dengan stok habis; test <i>idempotency</i> .
Seller dapat mengelola katalog produk secara mandiri.	FR-06, FR-07, FR-22, NFR-03, NFR-08	Test CRUD produk <i>seller</i> ; test akses ke produk <i>seller</i> lain; test diskon produk.
Transaksi keuangan terintegrasi dengan SmartBank via API Gateway.	FR-09, FR-10, FR-12, BR-01, BR-03, NFR-05, NFR-07	Test <i>checkout</i> sukses dan gagal; test kalkulasi <i>fee</i> ; test <i>rate limiting</i> ; simulasi SmartBank <i>offline</i> .
Pengiriman terkoordinasi dengan LogistiKita.	FR-11, R-03, NFR-07	Test <i>trigger</i> pengiriman setelah <i>payment</i> sukses; simulasi LogistiKita <i>offline</i> ; verifikasi <i>tracking_id</i> tersimpan.
Admin dapat mengelola platform dan memantau <i>analytics</i> .	FR-16, FR-19, FR-20, FR-21, NFR-09	Test ban/aktifkan <i>user</i> ; test <i>update</i> status <i>order</i> ; test <i>analytics endpoint</i> ; verifikasi <i>audit log</i> .
Sistem tetap tangguh saat <i>dependency</i> eksternal gagal.	NFR-06, NFR-07, R-03, R-04	Matikan <i>mock</i> SmartBank/LogistiKita; verifikasi <i>error handling</i> ; verifikasi <i>integration_logs</i> tercatat.
Data transaksi terlindungi dan teraudit.	NFR-01, NFR-03, NFR-09, NFR-12, R-05, R-06	Test akses lintas <i>user</i> ; verifikasi <i>password_hash</i> tidak <i>ter-expose</i> ; verifikasi <i>snapshot</i> harga <i>immutable</i> .

LAMPIRAN A

GLOSARIUM

Tabel 16: Glosarium Istilah SRS PasarKita

Istilah	Definisi Operasional
PasarKita	Platform <i>marketplace</i> digital B2C untuk produk UMKM; <i>Demand Generator</i> dalam ekosistem RPL 2.
Ekosistem UMKM RPL 2	Kumpulan <i>microservices</i> antar kelompok yang saling terintegrasi: SmartBank, WarungPOS, SupplierHub, LogistiKita, UMKM Insight, dan API Gateway.
API Gateway	<i>Service</i> Kelompok 7 yang bertindak sebagai <i>routing layer</i> , <i>JWT validator</i> antar- <i>service</i> , dan <i>logging</i> inter- <i>service</i> .
SmartBank	<i>Service</i> Kelompok 1 sebagai <i>core</i> keuangan; memproses pembayaran dan mengelola saldo <i>user</i> .
LogistiKita	<i>Service</i> Kelompok 5 yang mengelola pengiriman dan menghasilkan <i>tracking_id</i> .
<i>Demand Generator</i>	Peran PasarKita sebagai inisiator permintaan (<i>buyer</i> melakukan <i>checkout</i>) dalam ekosistem.
<i>Idempotency Key</i>	UUID unik yang dikirimkan pada setiap <i>checkout request</i> untuk mencegah pemrosesan duplikat.
<i>Snapshot Harga</i>	Nilai <i>price_at_purchase</i> pada <i>order_items</i> yang diambil saat transaksi dan tidak berubah meskipun harga produk diperbarui.
<i>Soft Delete</i>	Penghapusan logis dengan mengubah <i>is_active</i> = <i>false</i> tanpa menghapus <i>record</i> dari <i>database</i> .
<i>Fee Marketplace</i>	Biaya platform sebesar 2% dari <i>subtotal order</i> yang dibebankan ke <i>buyer</i> setiap transaksi.
<i>Mock Server</i>	Implementasi lokal di <i>mock/</i> yang mensimulasikan SmartBank dan LogistiKita untuk keperluan <i>development</i> tanpa bergantung pada <i>service</i> nyata.
<i>System of Record</i>	Sistem otoritatif untuk satu jenis data. PasarKita adalah <i>system of record</i> untuk produk, <i>order</i> , dan ulasan dalam ekosistemnya; SmartBank adalah <i>system of record</i> untuk saldo dan transaksi keuangan.

LAMPIRAN B

REFERENSI INTERNAL

Tabel 17: Referensi Internal Artefak Proyek PasarKita

Artefak	Relevansi
README.md	Deskripsi umum aplikasi, arsitektur ekosistem, <i>tech stack</i> , API <i>endpoints</i> , <i>role</i> & akses, dan aturan keuangan ekosistem.
backend/database/schema/000_full_schema.sql	Skema lengkap semua tabel Supabase PostgreSQL: <i>users</i> , <i>products</i> , <i>orders</i> , <i>order_items</i> , <i>ratings</i> , <i>notifications</i> , <i>vouchers</i> , <i>integration_logs</i> , <i>admin_audit_logs</i> , dan lainnya.
backend/src/modules/	<i>Business logic</i> per domain: <i>auth</i> , <i>products</i> , <i>orders</i> , <i>checkout</i> , dan <i>integrations</i> .
backend/src/middlewares/	<i>Middleware</i> <i>auth</i> (JWT validation), <i>validate</i> (Zod <i>schema</i>), dan <i>errorHandler</i> (<i>centralized error response</i>).
backend/src/integrations/	Adapter HTTP untuk SmartBank dan LogistiKita; konfigurasi GATEWAY_BASE_URL vs <i>mock</i> URL.
backend/src/utills/fee.js	Implementasi kalkulasi <i>fee marketplace</i> 2%; digunakan oleh <i>checkout module</i> dan <i>fee simulation endpoint</i> .
backend/vercel.json	Konfigurasi <i>routing</i> Vercel untuk <i>backend serverless</i> ; memetakan semua <i>path</i> ke <i>api/index.js</i> .
frontend/store/	Zustand <i>store</i> untuk <i>auth state</i> (<i>user</i> , <i>token</i> , <i>login/logout</i>) dan <i>cart state</i> (<i>items</i> , <i>add</i> , <i>remove</i> , <i>clear</i>).
mock/smartbank/ dan mock/logistikita/	Implementasi <i>mock server</i> lokal; digunakan hanya saat NODE_ENV=development.

LAMPIRAN C

CATATAN PERUBAHAN VERSI

Tabel 18: Catatan Perubahan Versi Dokumen SRS

Versi	Tanggal	Keterangan
1.0	[Tanggal]	<i>Initial baseline SRS</i> ; disusun dari README, skema <i>database</i> , dan struktur kode aktif. Mencakup semua bagian: pendahuluan, gambaran produk, arsitektur, domain data, kebutuhan fungsional (FR-01 s/d FR-22), kebutuhan non-fungsional (NFR-01 s/d NFR-12), antarmuka eksternal, <i>workflow</i> operasional, risiko, dan <i>traceability matrix</i> .